Softmax

- Multiclass classifier.

$$h(x) = \frac{1}{\sum_{j=1}^{k} e^{\theta_j \cdot x/r - c}} \begin{bmatrix} e^{\theta_1 \cdot x/r - c} \\ e^{\theta_2 \cdot x/r - c} \\ \vdots \\ e^{\theta_k \cdot x/r - c} \end{bmatrix}$$

$r > 0$ is the temperature parameter.

$$c = \begin{bmatrix} \max_l [\theta_j \cdot x_l / r] \\ \vdots \end{bmatrix}$$

$$P(y^{(i)} = j \mid x^{(i)}, \theta) = \frac{e^{\theta_j \cdot x^{(i)}/r}}{\sum_{l=0}^{k-1} e^{\theta_l \cdot x^{(i)}/r}}$$

Representation of data

7. Classification Using Manually Crafted Features

Bookmark this page

The performance of most learning algorithms depends heavily on the representation of the training data. In this section, we will try representing each image using different features in place of the raw pixel values. Subsequently, we will investigate how well our regression model from the previous section performs when fed different representations of the data.

Dimensionality Reduction via PCA

Principal Components Analysis (PCA) is the most popular method for linear dimension reduction of data and is widely used in data analysis. For an in-depth exposition see: https://online.stat.psu.edu/stat505/lesson/11.

Briefly, this method finds (orthogonal) directions of maximal variation in the data. By projecting an $n \times d$ dataset X onto $k \leq d$ of these directions, we get a new dataset of lower dimension that reflects more variation in the original data than any other k-dimensional linear projection of X. By going through some linear algebra, it can be proven that these directions are equal to the k eigenvectors corresponding to the k largest eigenvalues of the covariance matrix $\widetilde{X}^T \widetilde{X}$, where $\widetilde{X}$ is a centered version of our original data.

Remark: The best implementations of PCA actually use the Singular Value Decomposition of $\widetilde{X}$ rather than the more straightforward approach outlined here, but these concepts are beyond the scope of this course.

Cubic Features

In this section, we will also work with a **cubic feature** mapping which maps an input vector $x = [x_1, \ldots, x_d]$ into a new feature vector $\phi(x)$, defined so that for any $x, x' \in \mathbb{R}^d$:

$$\phi(x)^T \phi(x') = (x^T x' + 1)^3$$